

Create AGENTS.md in your root. This file teaches all AI agents (Claude, Copilot, Gemini) how to use your specific Spec-Kit workflow.

```
```markdown
```

```
AGENTS.md
```

Here is a **significantly improved, clearer, more actionable, more valuable** version of your **AGENTS.md**.

I kept the spirit but made it **practical**, **strict**, and **agent-compatible**, so Claude/Gemini/Copilot can actually follow it in real workflows.

```

```

```
AGENTS.md
```

```
Purpose
```

This project uses **Spec-Driven Development (SDD)** — a workflow where **no agent is allowed to write code until the specification is complete and approved**.

All AI agents (Claude, Copilot, Gemini, local LLMs, etc.) must follow the **Spec-Kit lifecycle**:

```
> Specify → Plan → Tasks → Implement
```

This prevents “vibe coding,” ensures alignment across agents, and guarantees that every implementation step maps back to an explicit requirement.

```

```

```
How Agents Must Work
```

Every agent in this project **MUST** obey these rules:

1. **Never generate code without a referenced Task ID.**
2. **Never modify architecture without updating `speckit.plan`.**
3. **Never propose features without updating `speckit.specify` (WHAT).**
4. **Never change approach without updating `speckit.constitution` (Principles).**
5. **Every code file must contain a comment linking it to the Task and Spec sections.**

If an agent cannot find the required spec, it must **stop and request it**, not improvise.

```

```

```
Spec-Kit Workflow (Source of Truth)
```

```
1. Constitution (WHY — Principles & Constraints)
```

File: `speckit.constitution`

Defines the project’s non-negotiables: architecture values, security rules, tech stack constraints,

performance expectations, and patterns allowed.

Agents must check this before proposing solutions.

---

### \*\*2. Specify (WHAT — Requirements, Journeys & Acceptance Criteria)\*\*

File: `speckit.specify`

Contains:

- \* User journeys
- \* Requirements
- \* Acceptance criteria
- \* Domain rules
- \* Business constraints

Agents must not infer missing requirements — they must request clarification or propose specification updates.

---

### \*\*3. Plan (HOW — Architecture, Components, Interfaces)\*\*

File: `speckit.plan`

Includes:

- \* Component breakdown
- \* APIs & schema diagrams
- \* Service boundaries
- \* System responsibilities
- \* High-level sequencing

All architectural output MUST be generated from the Specify file.

---

### \*\*4. Tasks (BREAKDOWN — Atomic, Testable Work Units)\*\*

File: `speckit.tasks`

Each Task must contain:

- \* Task ID
- \* Clear description
- \* Preconditions

- \* Expected outputs
- \* Artifacts to modify
- \* Links back to Specify + Plan sections

Agents **implement only what these tasks define**.

---

### **5. Implement (CODE — Write Only What the Tasks Authorize)**

Agents now write code, but must:

- \* Reference Task IDs
- \* Follow the Plan exactly
- \* Not invent new features or flows
- \* Stop and request clarification if anything is underspecified

> The golden rule: **No task = No code.**

---

## **Agent Behavior in This Project**

### **When generating code:**

Agents must reference:

'''

[Task]: T-001

[From]: speckit.specify §2.1, speckit.plan §3.4

'''

### **When proposing architecture:**

Agents must reference:

'''

Update required in speckit.plan → add component X

'''

### **When proposing new behavior or a new feature:**

Agents must reference:

'''

Requires update in speckit.specify (WHAT)

'''

### \*\*When changing principles:\*\*

Agents must reference:

'''

Modify constitution.md → Principle #X

'''

---

## \*\*Agent Failure Modes (What Agents MUST Avoid)\*\*

Agents are NOT allowed to:

- \* Freestyle code or architecture
- \* Generate missing requirements
- \* Create tasks on their own
- \* Alter stack choices without justification
- \* Add endpoints, fields, or flows that aren't in the spec
- \* Ignore acceptance criteria
- \* Produce "creative" implementations that violate the plan

If a conflict arises between spec files, the \*\*Constitution > Specify > Plan > Tasks\*\* hierarchy applies.

---

## \*\*Developer-Agent Alignment\*\*

Humans and agents collaborate, but the \*\*spec is the single source of truth\*\*.  
Before every session, agents should re-read:

1. ``.memory/constitution.md``

This ensures predictable, deterministic development.

'''